

LLD Practice Platform — Design Note

1. MVP Scope

The MVP is intentionally focused on the core LLD learning loop rather than building a complete learning management or assessment system. The goal is to let a learner choose an LLD problem, submit a design, receive explainable feedback, review the attempt, and retry.

Included capabilities

- 3-5 LLD problems with descriptions, requirements, constraints, difficulty, and relevant context.
- A practice attempt associated with a selected problem.
- Structured text-based submission covering approach, classes/responsibilities, relationships, abstractions/interfaces, patterns, assumptions, and edge cases.
- Submission status tracking and persistence before evaluation.
- Deterministic validation for required submission sections.
- Rubric-based qualitative evaluation using eight LLD criteria.
- LLM-assisted feedback with evidence, concerns, suggestions, and confidence.
- Attempt history, result review, retry, and evaluation-failure handling.
- Tests for important validation and evaluation behavior.

2. User Flow

Choose Problem → Understand → Design → Submit → Get Feedback → Review → Try Again

On the home page, the learner sees problem cards and selects a problem. The problem page presents the description, requirements, constraints, and context. Starting a problem creates an Attempt record.

The practice screen provides structured fields for the learner's design. The learner can save a draft while working. On submission, the platform validates the required sections, persists the submission, changes the attempt state, and starts evaluation.

The evaluation page presents an overall result and criterion-level feedback. The learner can return to the same problem, view the submitted design, or retry after a failed evaluation. History provides access to earlier attempts.

3. Domain Model

- Problem — stores the LLD challenge, difficulty, description, requirements, and constraints.
- Attempt — represents one learner practice session for a problem and owns its lifecycle state.
- Submission — stores the learner's actual design response for an attempt.
- Evaluation — stores the evaluator type, evaluation state, rubric criteria, feedback, and evaluation metadata.
- Evaluator — an abstraction defining the evaluation contract so the practice flow does not depend directly on Gemini or another evaluator.

4. Relationships

Entity	Relationship	Purpose
Problem	1 → many Attempts	A problem can be practiced repeatedly.
Attempt	1 → 1 Submission	An attempt contains one submitted design.
Attempt	1 → 1 Evaluation	An attempt receives one evaluation result for its current submission.
Evaluator	implemented by evaluators	Keeps evaluation replaceable.

LLD Practice Platform — Design Note

5. Evaluation Design

LLD evaluation cannot be reduced to comparing a submission with one reference implementation because multiple designs can satisfy the same requirements. The evaluator therefore uses a fixed rubric and evaluates the learner's reasoning and design evidence.

Eight rubric criteria

- Requirement Understanding
- Class Responsibilities
- Coupling & Cohesion
- Encapsulation & Interfaces
- Abstraction & Patterns
- Extensibility
- Edge Cases & Testability
- Explanation Quality

The feedback model is structured as criterion → score → evidence → concern → suggestion → confidence. This makes the result more useful than a single numerical score and gives the learner a clear reason for each evaluation.

6. Deterministic Validation + LLM Evaluation

Deterministic layer: validates that the problem and submission exist, required sections are present, submission state is valid, and the evaluation response follows the expected structure. This provides predictable behavior for basic correctness.

LLM layer: evaluates judgment-heavy areas such as requirement interpretation, responsibility assignment, abstraction, coupling/cohesion, extensibility, edge cases, and improvement suggestions. Gemini is used behind the Evaluator abstraction.

The LLM response is requested in a structured format and validated before it is persisted. Invalid or incomplete evaluation responses are treated as failures rather than silently being shown as valid feedback.

7. Evaluation State and Failure Handling

Evaluation is modeled as an explicit state transition so slow or failed evaluation does not make the practice flow ambiguous.

State	Meaning
DRAFT	Learner can edit and save the current submission.
SUBMITTED	Submission has been persisted and is ready for evaluation.
EVALUATING	An evaluator is processing the submission.
COMPLETED	Evaluation has been successfully persisted.
FAILED	Evaluation failed; the learner can retry without recreating the submission.

The submission is persisted before evaluation begins. If the evaluator fails, the Attempt becomes FAILED and the user can retry evaluation directly. This avoids losing learner work and separates submission persistence from evaluator availability.

8. Explainable Feedback

The evaluation is designed to answer not only “What score did I get?” but also “Why?” and “What should I change?” Each criterion includes evidence from the learner's submission, a concern where applicable, a concrete improvement suggestion, and evaluator confidence.

This approach also supports multiple valid solutions: feedback focuses on the quality of the learner's reasoning against the rubric instead of requiring an exact class structure or pattern choice.

LLD Practice Platform — Design Note

9. Extensibility

The design keeps the core practice flow independent from the representation and evaluation mechanism. This is achieved through clear domain boundaries and an Evaluator abstraction.

Change Test A — Text to Class Diagram

The MVP accepts structured text, but a future version may accept a class diagram. The Practice/Attempt flow should not need to change because the learner still creates an attempt, submits a design, receives feedback, and reviews the result. The submission representation can evolve while the core domain remains stable.

Change Test B — LLM to Rule-Based or Human Evaluation

The practice flow should also remain stable if Gemini is replaced by a rule-based evaluator or human review. The Evaluator abstraction defines the contract, allowing another evaluator implementation to be introduced without rewriting the attempt lifecycle or frontend practice experience.

10. Architecture

The MVP uses a simple modular monolith because the assignment is primarily about LLD/domain design. The system is divided into a React client and an Express/MongoDB server with clear separation between routes, controllers, models, and evaluation services.

Client: provides problem selection, practice, submission, evaluation results, and history screens.

Server: exposes REST endpoints and coordinates domain operations.

Persistence: MongoDB stores Problems, Attempts, Submissions, and Evaluations.

Evaluation services: validation, rubric definitions, response validation, and evaluator implementations are separated from the attempt controller.

11. Persistence and API Responsibilities

- Problem endpoints provide the available LLD challenges.
- Attempt endpoints create attempts, return history/details, save drafts, submit attempts, and trigger evaluation.
- Submission data is stored separately from the Attempt lifecycle so learner work remains persisted even when evaluation fails.
- Evaluation data stores structured rubric results and evaluator metadata.
- The frontend consumes these REST APIs and does not contain evaluation logic.

12. Trade-offs and Engineering Judgment

The main trade-off is simplicity versus future scale. A modular monolith is sufficient for the current workflow and keeps the codebase understandable. Introducing microservices, Kafka, Redis, Kubernetes, or multi-region infrastructure would add operational complexity without improving the core learner experience at MVP scale.

LLM evaluation is useful for qualitative judgment but is not fully deterministic. The system therefore combines deterministic validation with structured LLM output validation and explicit FAILED/retry handling.

Structured text is another deliberate MVP trade-off. It captures enough information to evaluate important LLD concepts while avoiding the complexity of building a diagram editor or code execution environment within the assignment scope.

13. Limitations and Future Improvements

- No complex authentication or user-management system in the MVP.
- No full UML/class-diagram editor or code execution environment.
- LLM feedback can vary and should be improved with evaluator calibration and testing.
- Future evaluators could include rule-based checks, human review, or other models.
- Future submissions could support code, UML/class diagrams, or combined text and diagram formats.
- If usage grows, evaluation jobs could later be moved to asynchronous processing without changing the core domain model.

14. Final Design Principle

The design prioritizes a small, reliable learning loop: practice → submit → explainable feedback → review → retry. The domain model and evaluator abstraction keep the MVP simple while leaving clear seams for future submission formats and evaluation strategies.